LLMBilliardsMaster: A Hierarchical Framework for Billiards with Large Language Models

Kaiyang Xu[1,†] and Han Wu[1]

***Abstract*— While Large Language Models (LLMs) excel at strategic reasoning, they struggle with the precise continuous control required for physics-based games like billiards. To address this, we propose LLMBilliardsMaster, a hierarchical framework that grounds semantic planning in physical reality. Our approach introduces two key modules: (1) Geometry Module, which provides theoretical "Ghost Ball" aiming hints to guide the LLM; and (2) a Feedback & Replan Module that iteratively self-corrects physical errors before execution. Experiments across multiple foundation models demonstrate that our framework improves the average score from 27.5 to 61.25 (on GLM-4.5). These results confirm that synergizing the semantic intelligence of LLMs with the geometric precision of traditional solvers is the optimal path for robust robotic manipulation.**

I. INTRODUCTION

Billiards stands as a quintessential benchmark for embodied intelligence, demanding a unique synthesis of long-horizon strategic planning and precise continuous control. Unlike board games such as chess or Go, billiards involves continuous action spaces and requires accurate modeling of physical interactions between balls, cushions, and pockets. This makes it an ideal testbed for developing AI agents that can handle real-world physical constraints while making strategic decisions.

Traditional approaches typically rely on Reinforcement Learning (RL) or search-based methods such as MCTS [1]. While search-based algorithms excel at low-level execution precision, they suffer from the "curse of dimensionality" when the search space is vast. Conversely, Large Language Models (LLMs) have demonstrated impressive capabilities in high-level reasoning and semantic understanding. However, applying LLMs directly to continuous control tasks presents a fundamental paradox: LLMs naturally struggle with precise numerical computation but possess significant potential in high-level decision-making. They often hallucinate unrealistic physical parameters [2], failing to bridge the gap between semantic intent and continuous execution.

To address these challenges, we draw inspiration from recent works in robotic manipulation, such as RoCo [3], which decouple high-level logic from low-level control. We propose **LLMBilliardsMaster**, a hierarchical framework that integrates the strategic reasoning of LLMs with the precision of traditional optimization algorithms.

As illustrated in Fig. 1, the hierarchical framework adopts a "coarse-to-fine" strategy consisting of three distinct levels:

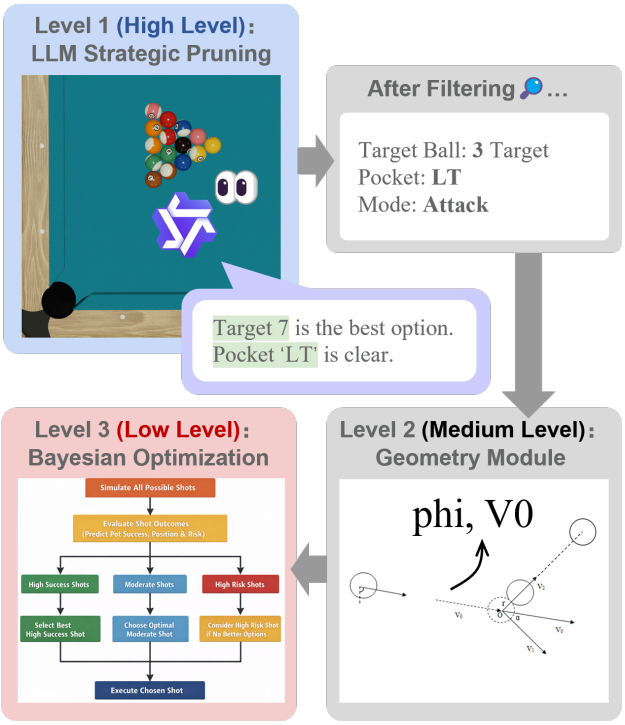

Fig. 1. **The hierarchical framework of LLMBilliardsMaster. High Level:** The LLM analyzes the global state to issue a strategic directive, greatly pruning the search space. **Medium Level:** The Geometry Module calculates a theoretical initial guess. **Low Level:** The Bayesian Optimization Module performs fine-grained local search to ensure execution precision.

1) **Level 1: Strategic Pruning (The Eyes).** The LLM serves as the high-level planner. By semantically analyzing the global table state, it identifies the optimal target ball and pocket, effectively filtering out approximately 80% of the invalid search space.
2) **Level 2: Geometric Grounding (The Brain).** To compensate for the LLM's lack of computational precision, we introduce a Geometry Module. This module calculates the "Ghost Ball" position based on the strategic directive, providing a high-quality *Initial Guess* for the kinematic parameters.
3) **Level 3: Precise Execution (The Hand).** Finally, a Bayesian Optimization module performs local fine-tuning around the geometric initial guess. This ensures robust physical execution even under simulation noise, achieving pixel-level precision.

Our main contributions are summarized as follows:

- We propose a LLM-based billiards agent, equipped with a "Think-Simulate-Reflexion-Execute" loop. By

*These authors contributed equally to this work.

† Han Wu and Kaiyang Xu contributed equally to this project; while this manuscript was primarily written by Kaiyang Xu.

[1]Shanghai Jiao Tong University, Shanghai, China

utilizing simulation feedback as a critic, the agent can iteratively self-correct its actions, significantly reducing foul rates.
- We introduce a Geometry Module within our framework that grounds the LLM's reasoning in physical constraints, translating raw continuous coordinates into semantic path analysis.
- We demonstrate our work achieves state-of-the-art performance across multiple LLMs such as DeepSeek and Qwen. Ablation studies confirm that synergizing semantic planning with geometric optimization significantly outperforms standalone methods.

II. RELATED WORK

A. AI in Games

Game-playing AI has been a central topic in artificial intelligence research since its inception. Early successes include Deep Blue's victory over world chess champion Garry Kasparov [4] and more recently, AlphaGo's defeat of world Go champion Lee Sedol [5]. These achievements primarily relied on tree search algorithms combined with evaluation functions or neural networks.

For games with continuous action spaces, reinforcement learning approaches have shown promise. Deep Q-Networks (DQN) [6] and policy gradient methods [7] have been successfully applied to video games and robotic control tasks. However, these approaches typically require extensive training data and computational resources.

B. Hierarchical LLM-Driven Decision Making

Large Language Models (LLMs) have demonstrated impressive capabilities in high-level reasoning and strategic planning across various domains. Following the dialectic framework of RoCo, recent research has further expanded LLM-based multi-agent collaboration. Chen et al. [8] investigated the trade-offs between centralized and decentralized architectures for scalable multi-robot planning. COHERENT [8] extended LLM collaboration to heterogeneous systems, utilizing a hierarchical framework for task allocation. Similarly, MALMM [9] proposed a multi-agent framework that distributes high-level planning and low-level control across specialized LLM agents to achieve zero-shot manipulation. These works collectively validate the effectiveness of decomposing complex tasks into hierarchical neuro-symbolic processes.

However, applying LLMs directly to physics-based games like billiards remains challenging due to their inherent lack of physical grounding and numerical computation ability. Our work addresses this limitation by grounding LLM reasoning within a hierarchical framework.

C. Traditional Geometry-Based Optimization Algorithms

To ensure precise low-level execution, our framework integrates the optimization-based agents developed by Han Wu [10], who presents a Geometry Module to compute theoretical "Ghost Ball" aim points and assess shot difficulty.

Building on this, the **NewAgent** employs Geometry-Guided Bayesian Optimization [11], which initializes the search with geometric solutions and constrains the exploration space to efficiently fine-tune continuous parameters (V_0, ϕ).

Complementarily, the **MCTSAgent** adapts Monte Carlo Tree Search by generating candidate actions around geometric baselines to balance exploration and exploitation.

Finally, the **EnsembleVotingAgent** aggregates these strategies via a simulation-based majority voting mechanism, achieving a 86.67% win rate against *BasicAgent* and a 60.83% win rate against *BasicAgentPro*, thereby serving as a robust execution engine for our hierarchical framework.

III. PROBLEM FORMULATIO

A. Game Environment

We consider the standard eight-ball pool game played on a regulation table with six pockets. The environment consists of 16 balls: one white cue ball, seven solid balls (1-7), seven striped balls (9-15), and one black eight ball. The agent must legally pocket its designated balls before the eight ball.

B. Hierarchical Action Space

Unlike traditional approaches that treat billiards purely as a continuous control problem, we formulate the decision process as a **bi-level optimization problem**, decomposing the action space into high-level strategic directives and low-level control primitives.

1) High-Level Strategic Space ($\mathscr{A}_{high}$): The high-level planner (LLM) selects a discrete strategic instruction $z_t \in \mathscr{A}_{high}$:

$$z_t = (\text{TargetID}, \text{PocketID}, \text{ShotType}) \tag{1}$$

where $\text{TargetID} \in \{1, \ldots, 15\}$ and $\text{PocketID} \in \{1, \ldots, 6\}$. This discrete abstraction reduces the planning horizon complexity by focusing on "what to do" rather than "how to do it."

2) Low-Level Control Space ($\mathscr{A}_{low}$): The low-level executor maps the strategic instruction z_t to a precise continuous control vector $\mathbf{u}_t \in \mathscr{A}_{low} \subset \mathbb{R}^5$:

$$\mathbf{u}_t = (V_0, \phi, \theta, a, b) \tag{2}$$

where $V_0 \in [0.5, 8.0]$ m/s is the velocity, $\phi \in [0, 360)^\circ$ is the aiming angle, and (a, b) are spin offsets.

IV. METHODOLOGY

We propose a **Geometric LLM Agent** designed to bridge the gap between semantic reasoning and continuous control in billiards. Unlike traditional methods that rely solely on numerical optimization, our approach models the decision-making process as a language-driven "Think-Simulate-Reflexion-Execute" closed loop [12] [13]. As illustrated in Fig. 2, the system comprises three core components: Prompt Engineering, LLM Reasoning, and Simulation-Based Reflexion.

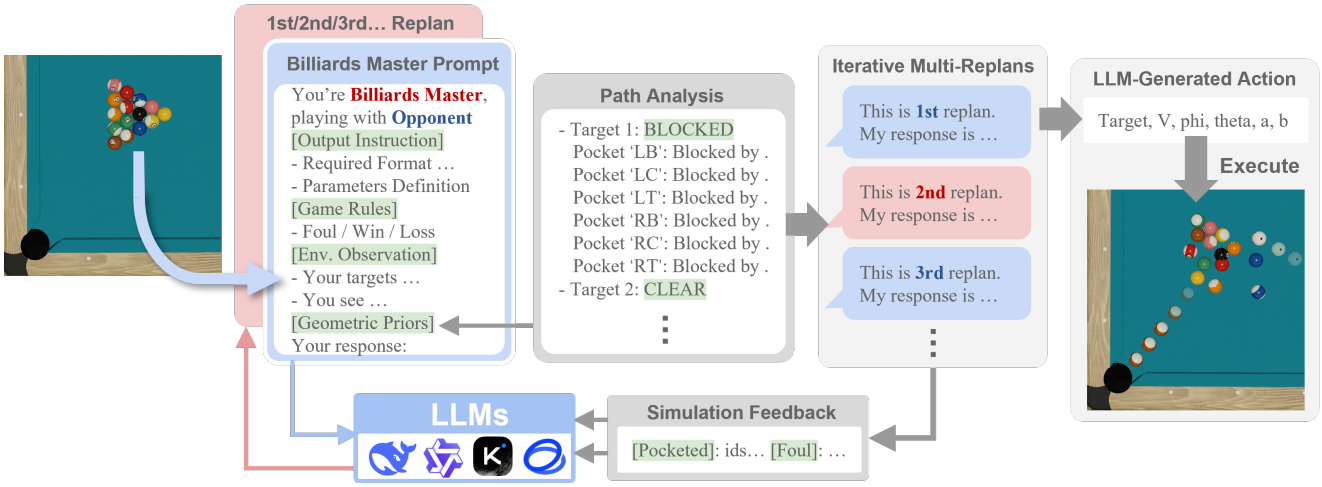

Fig. 2. **Overview of our proposed LLM Billiards Master.** The framework consists of four key modules: (1) **Prompt Module** transforms physical states into semantic observations and injects geometric priors; (2) **Geometry Module** generates the analysis of path and the position of ghost ball; (3) **Feedback & Replan Module** validates actions via physics simulation, providing natural language feedback for self-correction; (4) **Parse Module** converts the finalized text output into executable commands.

A. Prompt Engineering

To enable the Large Language Model (LLM) to perceive and reason about the continuous physical environment, we design a structured prompt engine that translates the raw game state s_t into a semantic context C_t. The prompt is composed of three distinct segments:

1) Output Instruction and Game Rules: We explicitly define the agent's role (e.g., "You are a Billiards Master") and enforce strict output formatting constraints to ensure the generated actions are parsable. Standard 8-ball rules are injected into the context to prevent rule-based violations, such as hitting the opponent's ball first or potting the 8-ball prematurely.

2) Observation and Path Analysis: Raw coordinates are insufficient for LLMs to understand spatial relationships. Therefore, we introduce a **Path Analysis** generated by Geometry Module that pre-processes the table state to generate semantic descriptions. For every target ball B_i and pocket P_k, we cast geometric rays to detect obstructions. The state is then described linguistically, for example:

*"Target 1 to Pocket LT: **BLOCKED** by Ball 7."*
*"Target 2 to Pocket RT: **CLEAR**."*

This semantic mapping allows the LLM to effectively prune invalid shots at the reasoning level.

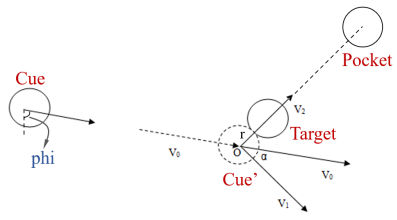

Fig. 3. **"Ghost Ball" algorithm.** The diagram illustrates how the ideal aiming angle ϕ is derived by targeting the center of a virtual "ghost ball" adjacent to the target ball.

3) Geometric Priors: Although LLMs excel at strategy, they struggle with precise geometric calculations. To mitigate this, we inject **Geometric Priors** derived from the "Ghost Ball" algorithm shown in Fig. 3, directly into the prompt. Instead of asking the LLM to guess the aiming angle ϕ from scratch, we provide the theoretical ideal values as reference hints. This prompts the LLM to focus on strategic adjustments rather than solving inverse kinematics.

B. Simulation-Based Reflexion Mechanism

A key contribution of our work is the **Feedback & Replan Module**, which acts as a "critic" to prevent hallucinations and physical errors. Inspired by the "inner monologue" concept in recent robotic agents [3], we implement a closed-loop feedback mechanism as shown in Fig. 4:

1) Pre-Execution Simulation: Before executing any action $\mathbf{u}_t$ proposed by the LLM on the real environment, the system creates a parallel simulation instance. The proposed shot is executed in this sandbox environment to predict its outcome.

2) Error Translation and Feedback: If the simulation detects a foul (e.g., *Scratch*, *No-Contact*, or *Wrong-Ball*), the error is captured and translated into a natural language feedback prompt F_t. For instance:

- **Physics Error:** *"The cue ball velocity was too low ($V_0 = 4.5$), resulting in no contact."*
- **Textual Feedback:** *"Simulation Result: FOUL (No Contact). Your velocity was too low to reach the target. Please increase V_0."*

3) Iterative Replanning: This feedback F_t is appended to the dialogue history, prompting the LLM to "reflect" on its previous failure. The LLM then generates a revised plan $\mathbf{u}'_t$. This cycle continues until a valid, non-foul shot is generated or a maximum retry limit is reached. As shown in our experiments, this mechanism significantly reduces the foul rate by allowing the agent to self-correct physically impossible or risky commands.

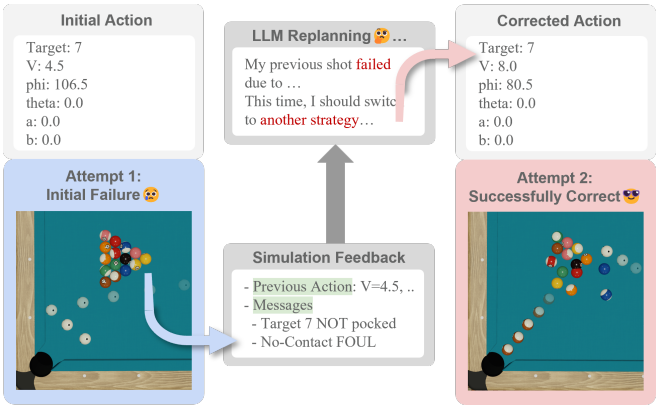

Fig. 4. **A Scenario for Feedback & Replan Module.** The agent initially proposes a shot with insufficient velocity ($V_0 = 4.5$) and improper angle ($\phi = 106.5$) , triggering a "No-Contact" foul (Left). Upon receiving this feedback, the agent self-corrects by correcting the velocity to 8.0 and ϕ to 80.5 (Right), successfully completing the shot.

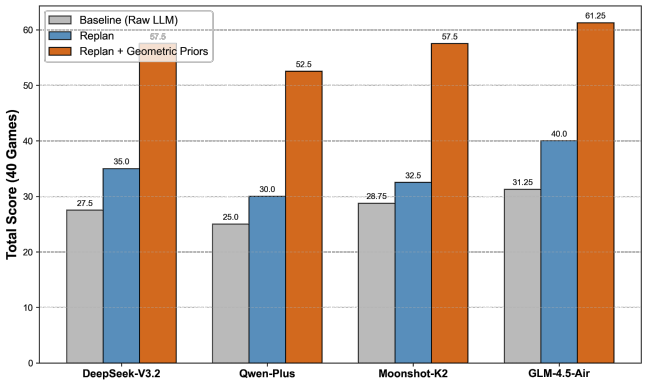

Fig. 5. **Ablation Study Results.** Comparison of Average Score across different system configurations. The introduction of Geometric Priors (Orange) yields the most significant performance boost compared to the Baseline (Gray) and Replan-only (Blue) variants.

V. EXPERIMENTS

A. Experimental Setup

We evaluate our method in a high-fidelity physics simulation environment based on `pooltool` [14], which models rigid body dynamics and continuous collisions for standard 8-ball pool. To ensure a fair comparison, all agents are tested under the same randomized initial conditions:

- **Test Set:** We conducted 40 experiments covering different turn orders (first vs. second player) and diverse shot types (solids vs. stripes) under randomized initial configurations.
- **Opponent:** All matches were played against the **BasicAgent**, a baseline opponent provided by the course environment that utilizes standard Bayesian Optimization without geometric guidance.
- **Models:** We evaluate performance across multiple Large Language Models, including **DeepSeek-V3.2**, **Qwen-Plus**, **MoonShot-K2** and **GLM-4.5-Air**, to assess the generalization capability of our prompt engineering.

B. Evaluation Metric

To provide a unified quantitative assessment, we define a single performance metric: **Average Score**. For each game i, the score S_i is calculated as:

$$S_i = \begin{cases} 1.0 & \text{if Win} \\ 0.5 & \text{if Draw} \\ 0.0 & \text{if Loss} \end{cases} \tag{3}$$

The final metric is the normalized average score (scaled to percentage): Score $= \left(\frac{1}{N}\sum_{i=1}^{N} S_i\right) \times 100$. This metric penalizes losses while giving partial credit for defensive stalemates, represented as a value between 0 and 100.

C. Ablation Study: Effectiveness of Modules

To validate the contribution of each component in our method, we conducted a comprehensive ablation study over three variants of our system:

1) **Baseline (Pure LLM):** The agent receives only raw coordinate observationsgame rules, and so on, without Geometry Module or FeedBack & Replan Module.
2) **Replan:** The agent includes the simulation-based feedback loop (Fig. 4) to self-correct fouls but lacks geometric priors.
3) **Replan + Geometry Priors (Ours):** The complete framework with both Geometry Module and FeedBack & Replan Module.

Analysis. As shown in Fig. 5, the **Baseline** achieves a low score (about 30) due to frequent foul-induced losses. The FeedBack & Replan Module improves the score to about 35 by filtering out physical errors. Finally, the injection of **Geometric Priors** boosts the score to about 60, demonstrating that guiding the LLM with "Ghost Ball" hints is crucial for winning.

D. Generalization Across Foundation Models

We extended the evaluation to multiple Large Language Models, to test cross-model robustness.

TABLE I
AVERAGE SCORE COMPARISON ACROSS LLMS

Model	Baseline Score (%)	Ours (Full) Score (%)
DeepSeek-V3.2	27.50	57.50
Qwen-Plus	25.00	52.50
MoonShot-K2	28.75	57.50
GLM-4.5-Air	31.25	**61.25**

Analysis. As indicated in Table I, our framework consistently improves the **Average Score** by a large margin (over 2.5×) across all tested models. This confirms that our Geometry Module and Feedback & Replan Module are a model-agnostic enhancer that effectively grounds semantic reasoning in physical reality.

E. The Necessity of Hierarchical Framework

To further contextualize our results, Table II compares our best-performing LLM agent (GLM-4.5-Air) against Han

Wu's traditional optimization agent (EnsembleVotingAgent) [10] on the same task.

TABLE II
COMPARISON: LLM-BASED VS. TRADITIONAL OPTIMIZATION AGENTS

Agent Types	Score (%)
EnsembleVotingAgent	**86.75**
LLMAgent	61.25

Analysis. As shown in Table II, although our framework significantly boosts LLM performance to a competitive level (Score 61.25), a performance gap remains compared to specialized algorithms (Score 86.75).

This discrepancy highlights a fundamental reality: **LLMs are semantically powerful but perform poorly as numerical solvers.** This finding strongly validates the premise of our proposed **Hierarchical Framework**. The optimal path forward is to decouple the roles: utilizing the LLM exclusively for high-level strategic planning while delegating the execution to robust traditional optimization algorithms.

VI. CONCLUSION

In this paper, we presented **LLMBilliardsMaster**, a physics-aware agent framework that enables Large Language Models to tackle the high-precision domain of billiards. By integrating a "Think-Simulate-Reflexion-Execute" loop with a structured prompt engine, we effectively bridge the gap between semantic reasoning and continuous control.

Our experiments yield three key insights:

1) **Geometric Grounding is Critical:** Raw LLMs struggle with physical hallucinations. The injection of "Ghost Ball" geometric priors serves as a necessary scaffold, greatly improving the average score across multiple foundation models.
2) **Reflexion Reduces Errors:** The simulation-based feedback loop allows the agent to iteratively self-correct physical violations, significantly reducing the foul rate without requiring parameter updates.
3) **The Necessity of Hierarchy:** Despite these improvements, a performance gap remains compared to specialized optimization algorithms. This confirms that LLMs are best suited as high-level strategic planners ("The Eyes"), while precise execution ("The Hand") should be delegated to robust numerical solvers.

Future work will focus on fully implementing the hierarchical interface to couple our LLM planner with the low-level Bayesian Optimization module, aiming to achieve human-level proficiency in billiards and broader robotic manipulation tasks.

REFERENCES

[1] C. Archibald, A. Altman, and Y. Shoham, "Computational pool: A new challenge for game theory theoreticians," *AI Magazine*, vol. 31, no. 4, pp. 33–41, 2010.

[2] K. Valmeekam, A. Olmo, S. Sreedharan, and S. Kambhampati, "On the planning abilities of large language models: A critical investigation," in *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.

[3] Z. Mandi, S. Jain, and S. Song, "RoCo: Dialectic Multi-Robot Collaboration with Large Language Models," Jul. 2023.

[4] M. Campbell, A. J. Hoane Jr, and F.-H. Hsu, "Deep blue," *Artificial Intelligence*, vol. 134, no. 1-2, pp. 57–83, 2002.

[5] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. Van Den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot *et al.*, "Mastering the game of go with deep neural networks and tree search," *Nature*, vol. 529, no. 7587, pp. 484–489, 2016.

[6] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[7] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[8] K. Liu, Z. Tang, D. Wang, Z. Wang, X. Li, and B. Zhao, "Coherent: Collaboration of heterogeneous multi-robot system with large language models," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025, pp. 10 208–10 214.

[9] H. Singh, R. J. Das, M. Han, P. Nakov, and I. Laptev, "Malmm: Multi-agent large language models for zero-shot robotics manipulation," *arXiv preprint arXiv:2411.17636*, 2024.

[10] H. Wu and K. Xu, "Ai agents for strategic billiards play: A comparative study of geometric heuristics, optimization methods, and large language models," Shanghai Jiao Tong University, Course Project Report, 2026, final Project for AI3603 Course.

[11] B. Shahriari, K. Swersky, Z. Wang, R. P. Adams, and N. De Freitas, "Taking the human out of the loop: A review of bayesian optimization," *Proceedings of the IEEE*, vol. 104, no. 1, pp. 148–175, 2015.

[12] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.

[13] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023.

[14] E. W. Katz and L. Hickcox, "Pooltool: A python sandbox for billiards simulation and analysis," *arXiv preprint arXiv:2403.03306*, 2024.